Project Documentation

File:	5506_ass.project
Date:	26/04/2026
Profile:	CODESYS V3.5 SP21 Patch 5

Table of contents

1 Device: Device 3
1.1 PLC Logic: Plc Logic 3
1.1.1 Application: Application 3
2 : Project Settings 22

1 Device: Device

Users and Groups

Users:

Groups

Access Rights

View
Modify
Execute
Add/remove children

Symbol Rights

Information

Name:	CODESYS Control Win V3 x64
Vendor:	CODESYS
Categories:	PLCs
Type:	4096
ID:	0000 0004
Version:	3.5.21.50
Order number:	???
Description:	CODESYS V3 Soft-PLC for Windows with non realtime capabilities (CODESYS Control Win V3 x64)

1.1 PLC Logic: Plc Logic

1.1.1 Application: Application

1.1.1.1 Global Variable List: GVL_IO

```
1   VAR_GLOBAL
2       // ========================
3       // Input devices
4       // ========================
5       Start_PB             :  BOOL ;
6       Stop_PB              :  BOOL ;
7       Reset_PB             :  BOOL ;
8       Abort_PB             :  BOOL ;
9
10      Infeed_Prox          :  BOOL ;
11      Fill_Pos_Sensor      :  BOOL ;
12      Level_Full_Sensor    :  BOOL ;
13      Red_Container_Sensor :  BOOL ;
14      Blue_Container_Sensor :  BOOL ;
15      Exit_Sensor          :  BOOL ;
16      Robot_Pick_Ready     :  BOOL ;
17
18      Conveyor_Fault       :  BOOL ;
19      Low_Fluid_A          :  BOOL ;
20      Low_Fluid_B          :  BOOL ;
21      Valve_A_FB           :  BOOL ;
22      Valve_B_FB           :  BOOL ;
23      Infeed_Block_Sensor  :  BOOL ;
24      Outfeed_Block_Sensor :  BOOL ;
25      Zero_Speed_SW        :  BOOL ;
26
27      // ========================
28      // Output devices
29      // ========================
30      Conveyor_Motor       :  BOOL ;
31      Valve_A              :  BOOL ;
32      Valve_B              :  BOOL ;
33      Pickup_Ready         :  BOOL ;
34      Alarm_Lamp           :  BOOL ;
35      Stack_Light_Green    :  BOOL ;
36      Stack_Light_Red      :  BOOL ;
37      Stack_Light_Amber    :  BOOL ;
38
39      // ========================
40      // Internal PackML variables
41      // ========================
42      SC                   :  BOOL ;
43      BatchDone            :  BOOL ;
44      ClearCmd             :  BOOL ;
45      HoldCmd              :  BOOL ;
46      UnholdCmd            :  BOOL ;
47      SuspendCmd           :  BOOL ;
48      UnsuspendCmd         :  BOOL ;
49      ExecStep             :  INT ;
50  END_VAR
51
```

1.1.1.2 Library Manager: Library Manager

```
#IoStandard
    Base Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    CmpErrors2 Interfaces, * (System)
    #CmpIoMgr Implementation
        CmpErrors2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpIoMgr Interfaces, * (System)
        Base Interfaces, * (System)
    CmpIoMgr Interfaces, * (System)
#3SLicense
    #CmpLog
        SysTypes2 Interfaces, * (System)
    #CAA Types
        #SysCpuHandling
            #SysMem
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #Standard
    #Component Manager
        #CmpEventMgr
            CmpEventMgr Interfaces, * (System)
            #CmpEventMgr Implementation
                CmpEventMgr Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
        #CmpApp
            SysTypes2 Interfaces, * (System)
            #CmpEventMgr
                CmpEventMgr Interfaces, * (System)
                #CmpEventMgr Implementation
                    CmpEventMgr Interfaces, * (System)
                    CmpErrors2 Interfaces, * (System)
                    SysTypes2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
    #CmpApp
        SysTypes2 Interfaces, * (System)
        #CmpEventMgr
            CmpEventMgr Interfaces, * (System)
            #CmpEventMgr Implementation
                CmpEventMgr Interfaces, * (System)
                CmpErrors2 Interfaces, * (System)
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #CmpCodeMeter
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
```

```
    CmpErrors2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    #SysCpuHandling
        #SysMem
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
    #CmpEventMgr
        CmpEventMgr Interfaces, * (System)
        #CmpEventMgr Implementation
            CmpEventMgr Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
#CAA Device Diagnosis
    #IoStandard
        Base Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
        #CmpIoMgr Implementation
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpIoMgr Interfaces, * (System)
            Base Interfaces, * (System)
        CmpIoMgr Interfaces, * (System)
    #CAA Types
        #SysCpuHandling
            #SysMem
                SysTypes2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
    #CmpEventMgr
        CmpEventMgr Interfaces, * (System)
        #CmpEventMgr Implementation
            CmpEventMgr Interfaces, * (System)
            CmpErrors2 Interfaces, * (System)
            SysTypes2 Interfaces, * (System)
        SysTypes2 Interfaces, * (System)
        CmpErrors2 Interfaces, * (System)
    IoDriver Interfaces, * (System)
    Base Interfaces, * (System)
    CmpErrors2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
    #CmpAsyncMgr
        SysTypes2 Interfaces, * (System)
    #CommFB
        #SysMem
            SysTypes2 Interfaces, * (System)
        IoDriver Profibus2 Interfaces, * (System)
#Standard
#BreakpointLogging
    #CmpLog
        SysTypes2 Interfaces, * (System)
    SysTypes2 Interfaces, * (System)
#IecSfc
    #Standard
```

1.1.1.3 POU: Aborted_ST

```
1   FUNCTION_BLOCK Aborted_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4   Pickup_Ready := FALSE ;
5
6   Alarm_Lamp := TRUE ;
7   Stack_Light_Green := FALSE ;
8   Stack_Light_Amber := FALSE ;
9   Stack_Light_Red := TRUE ;
10
11  SC := FALSE ;
12
```

1.1.1.4 POU: Aborting_ST

```
1   FUNCTION_BLOCK Aborting_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4   Pickup_Ready := FALSE ;
5
6   Alarm_Lamp := TRUE ;
7   Stack_Light_Green := FALSE ;
8   Stack_Light_Amber := FALSE ;
9   Stack_Light_Red := TRUE ;
10
11  SC := TRUE ;
12
```

1.1.1.5 POU: Clearing_ST

```
FUNCTION_BLOCK Clearing_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := FALSE ;
Stack_Light_Red := FALSE ;

ClearCmd := FALSE ;
SC := TRUE ;

```

1.1.1.6 POU: Complete_ST

```
FUNCTION_BLOCK Complete_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
Stack_Light_Green := FALSE ;
Stack_Light_Amber := TRUE ;
Stack_Light_Red := FALSE ;

SC := FALSE ;

```

1.1.1.7 POU: Completing_ST

```
FUNCTION_BLOCK Completing_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

SC := TRUE ;

```

1.1.1.8 POU: Execute_ST

```
FUNCTION_BLOCK Execute_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
CASE ExecStep OF

    0 :
        Conveyor_Motor := FALSE ;
        Valve_A := FALSE ;
        Valve_B := FALSE ;
        Pickup_Ready := FALSE ;
        BatchDone := FALSE ;

        IF Infeed_Prox THEN
            ExecStep := 10 ;
        END_IF ;

    10 :
        Conveyor_Motor := TRUE ;

        IF Fill_Pos_Sensor THEN
            Conveyor_Motor := FALSE ;
            ExecStep := 20 ;
        END_IF ;

    20 :
        IF Red_Container_Sensor THEN
            Valve_A := TRUE ;
```

```
25                  Valve_B  :=  FALSE ;
26              ELSIF  Blue_Container_Sensor  THEN
27                  Valve_A  :=  FALSE ;
28                  Valve_B  :=  TRUE ;
29              END_IF ;
30
31              IF  Level_Full_Sensor  THEN
32                  Valve_A  :=  FALSE ;
33                  Valve_B  :=  FALSE ;
34                  ExecStep  :=  30 ;
35              END_IF ;
36
37          30 :
38              Conveyor_Motor  :=  TRUE ;
39
40              IF  Exit_Sensor  THEN
41                  Conveyor_Motor  :=  FALSE ;
42                  Pickup_Ready  :=  TRUE ;
43                  ExecStep  :=  40 ;
44              END_IF ;
45
46          40 :
47              IF  Robot_Pick_Ready  THEN
48                  Pickup_Ready  :=  FALSE ;
49                  ExecStep  :=  50 ;
50              END_IF ;
51
52          50 :
53              BatchDone  :=  TRUE ;
54              ExecStep  :=  0 ;
55
56      END_CASE ;
57
58      // 外部条件，触发 Suspend
59      IF  Outfeed_Block_Sensor  THEN
60          SuspendCmd  :=  TRUE ;
61      ELSE
62          SuspendCmd  :=  FALSE ;
63      END_IF ;
64
65      // 严重故障时报警
66      IF  Conveyor_Fault  OR  Low_Fluid_A  OR  Low_Fluid_B  THEN
67          Alarm_Lamp  :=  TRUE ;
68          Stack_Light_Red  :=  TRUE ;
69      END_IF ;
70
```

1.1.1.9 POU: Held_ST

```
FUNCTION_BLOCK Held_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;

SC := FALSE ;

```

1.1.1.10 POU: Holding_ST

```
FUNCTION_BLOCK Holding_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;

SC := TRUE ;

```

1.1.1.11 POU: Idle_ST

```
FUNCTION_BLOCK Idle_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

Alarm_Lamp := FALSE ;
```

```
7       Stack_Light_Green  :=  FALSE ;
8       Stack_Light_Amber  :=  TRUE ;
9       Stack_Light_Red  :=  FALSE ;
10
11      SC  :=  FALSE ;
12
```

1.1.1.12 POU: PLC_PRG

```
1       PROGRAM  PLC_PRG
2       VAR
3       END_VAR
4
```

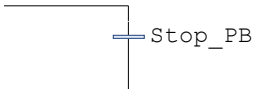
1.1.1.13 POU: Resetting_ST

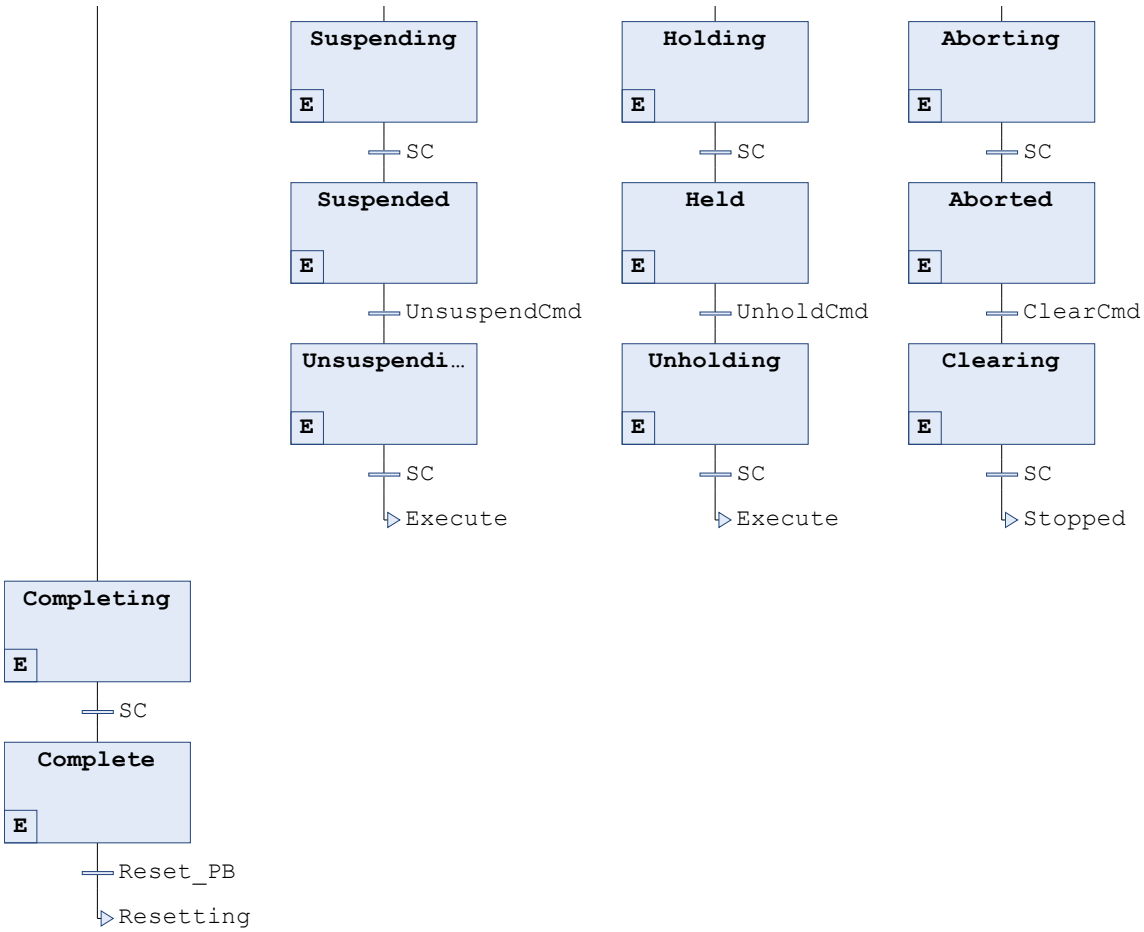
```
1       FUNCTION_BLOCK  Resetting_ST
2       VAR_INPUT
3       END_VAR
4       VAR_OUTPUT
5       END_VAR
6       VAR
7       END_VAR
8
```

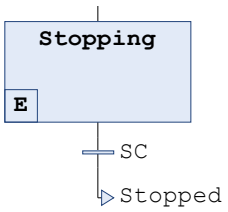
```
1       BatchDone  :=  FALSE ;
2       ClearCmd  :=  FALSE ;
3       HoldCmd  :=  FALSE ;
4       UnholdCmd  :=  FALSE ;
5       SuspendCmd  :=  FALSE ;
6       UnsuspendCmd  :=  FALSE ;
7
8       Conveyor_Motor  :=  FALSE ;
9       Valve_A  :=  FALSE ;
10      Valve_B  :=  FALSE ;
11      Pickup_Ready  :=  FALSE ;
12
13      Alarm_Lamp  :=  FALSE ;
14      Stack_Light_Green  :=  FALSE ;
15      Stack_Light_Amber  :=  FALSE ;
16      Stack_Light_Red  :=  FALSE ;
17
18      ExecStep  :=  0 ;
19
20      SC  :=  TRUE ;
21
```

1.1.1.14 POU: sfc

```
1   PROGRAM sfc
2   VAR
3       fbStopped        :  Stopped_ST ;
4       fbResetting      :  Resetting_ST ;
5       fbIdle           :  Idle_ST ;
6       fbStarting       :  Starting_ST ;
7       fbExecute        :  Execute_ST ;
8       fbCompleting     :  Completing_ST ;
9       fbComplete       :  Complete_ST ;
10      fbStopping       :  Stopping_ST ;
11      fbAborting       :  Aborting_ST ;
12      fbAborted        :  Aborted_ST ;
13      fbClearing       :  Clearing_ST ;
14      fbHolding        :  Holding_ST ;
15      fbHeld           :  Held_ST ;
16      fbUnholding      :  Unholding_ST ;
17      fbSuspending     :  Suspending_ST ;
18      fbSuspended      :  Suspended_ST ;
19      fbUnsuspending   :  Unsuspending_ST ;
20  END_VAR
21
```


Stop_PB


Suspending
E
SC
Suspended
E
UnsuspendCmd
Unsuspendi…
E
SC
Execute
Holding
E
SC
Held
E
UnholdCmd
Unholding
E
SC
Execute
Aborting
E
SC
Aborted
E
ClearCmd
Clearing
E
SC
Stopped
Completing
E
SC
Complete
E
Reset_PB
Resetting


Stopping
E
SC
Stopped

1.1.1.14.1 Action: _aAborted_entry

```
fbAborted ( ) ;

```

1.1.1.14.2 Action: _aAborting_entry

```
fbAborting ( ) ;

```

1.1.1.14.3 Action: _aClearing_entry

```
fbClearing ( ) ;

```

1.1.1.14.4 Action: _aComplete_entry

```
fbComplete ( ) ;

```

1.1.1.14.5 Action: _aCompleting_entry

```
fbCompleting ( ) ;

```

1.1.1.14.6 Action: _aExecute_entry

```
fbExecute ( ) ;

```

1.1.1.14.7 Action: _aHeld_entry

```
fbHeld ( ) ;

```

1.1.1.14.8 Action: _aHolding_entry

```
fbHolding ( ) ;

```

1.1.1.14.9 Action: _aIdle_entry

```
fbIdle ( ) ;

```

1.1.1.14.10 Action: _aResetting_entry

```
fbResetting ( ) ;

```

1.1.1.14.11 Action: _aStarting_entry

```
fbStarting ( ) ;

```

1.1.1.14.12 Action: _aStopped_entry

```
fbStopped ( ) ;

```

1.1.1.14.13 Action: _aStopping_entry

```
fbStopping ( ) ;

```

1.1.1.14.14 Action: _aSuspended_entry

```
fbSuspended ( ) ;

```

1.1.1.14.15 Action: _aSuspending_entry

```
fbSuspending ( ) ;

```

1.1.1.14.16 Action: _aUnholding_entry

```
fbUnholding ( ) ;

```

1.1.1.14.17 Action: _aUnsuspending_entry

1.1.1.15 POU: Starting_ST

```
FUNCTION_BLOCK Starting_ST
VAR_INPUT
END_VAR
VAR_OUTPUT
END_VAR
VAR
END_VAR

```

```
Conveyor_Motor := FALSE ;
Valve_A := FALSE ;
Valve_B := FALSE ;
Pickup_Ready := FALSE ;

```

```
6       IF NOT Conveyor_Fault
7          AND NOT Low_Fluid_A
8          AND NOT Low_Fluid_B THEN
9           Alarm_Lamp := FALSE ;
10          Stack_Light_Green := TRUE ;
11          Stack_Light_Amber := FALSE ;
12          Stack_Light_Red := FALSE ;
13          SC := TRUE ;
14      ELSE
15          Alarm_Lamp := TRUE ;
16          Stack_Light_Green := FALSE ;
17          Stack_Light_Amber := FALSE ;
18          Stack_Light_Red := TRUE ;
19          SC := FALSE ;
20      END_IF ;
21
```

1.1.1.16 POU: Stopped_ST

```
1       FUNCTION_BLOCK Stopped_ST
2       VAR_INPUT
3       END_VAR
4       VAR_OUTPUT
5       END_VAR
6       VAR
7       END_VAR
8
```

```
1       Conveyor_Motor := FALSE ;
2       Valve_A := FALSE ;
3       Valve_B := FALSE ;
4       Pickup_Ready := FALSE ;
5
6       Alarm_Lamp := FALSE ;
7       Stack_Light_Green := FALSE ;
8       Stack_Light_Amber := FALSE ;
9       Stack_Light_Red := FALSE ;
10
11      SC := FALSE ;
12
```

1.1.1.17 POU: Stopping

```
1       FUNCTION_BLOCK Stopping
2       VAR_INPUT
3       END_VAR
4       VAR_OUTPUT
5       END_VAR
6       VAR
7       END_VAR
8
```

1.1.1.18 POU: Stopping_ST

```
1   FUNCTION_BLOCK Stopping_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4   Pickup_Ready := FALSE ;
5
6   SC := TRUE ;
7
```

1.1.1.19 POU: Suspended_ST

```
1   FUNCTION_BLOCK Suspended_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4
5   SC := FALSE ;
6
```

1.1.1.20 POU: Suspending_ST

```
1   FUNCTION_BLOCK Suspending_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   Conveyor_Motor := FALSE ;
2   Valve_A := FALSE ;
3   Valve_B := FALSE ;
4
5   SC := TRUE ;
```

```
6
```

1.1.1.21 POU: Unholding_ST

```
1   FUNCTION_BLOCK Unholding_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   UnholdCmd := FALSE ;
2   SC := TRUE ;
3
```

1.1.1.22 POU: Unsuspending_ST

```
1   FUNCTION_BLOCK Unsuspending_ST
2   VAR_INPUT
3   END_VAR
4   VAR_OUTPUT
5   END_VAR
6   VAR
7   END_VAR
8
```

```
1   UnsuspendCmd := FALSE ;
2   SC := TRUE ;
3
```

1.1.1.23 Task Configuration: Task Configuration

Max. number of tasks: 100
Max. number of cyclic tasks: 100
Max. number of freewheeling tasks: 100
Max. number of event tasks: 100
Max. number of status tasks: 100

System Events:

1.1.1.23.1 Task: MainTask

Priority: 1
Type: Cyclic
Interval: t#20ms Unit: ms
Watchdog: Inactive
POUs: PLC_PRG

1.1.1.23.1.1 Program Call: PLC_PRG

2 : Project Settings

Static Analysis Light:

Unused variables (#33): 0
Overlapping memory areas (#28): 0
Write access from several tasks (#6): 0
Multiple write access on output (#4): 0
Multiple uses of identifiers (#27): 0
Report temporary FunctionBlock instances (#167): 0
Suspicious operation on string (#175): 0